

CONCORDIA UNIVERSITY
Department of Electrical and Computer Engineering
COEN 313: Digital Systems Design II Summer 2026

PROJECT: Sequential "3k+1" sequence generator with 7-segment hex display

Objectives

- To become well-versed in the art of register transfer level design using VHDL.
- To become acquainted with the algorithmic capability of a VHDL clocked process by designing a digital system which implements an algorithm using only a single clocked process.
- To become familiar with the ASM chart and control unit/datapath design methodology by implementing using this design methodology the same algorithm expressed by the 'single clocked process' method.
- To become familiar with the mathematical operators and data types contained within the package `numeric_std` of the IEEE library.
- To become familiar with time-multiplexed 7-segment LED displays and clock-divider digital circuitry.

Introduction

The "3k+1" sequence is well known in mathematics and is defined by the following:

Start with any positive integer k and apply the following rule to obtain the next term in the sequence:

```

if ( k is an even number )
    k = k / 2 ; (divide k by 2)
else
    k = 3*k + 1 ; (multiply k times 3 and add 1)

```

Keep applying this rule until the value of k reaches 1. The number of terms in the sequence is called its "length" [1]. Some typical example sequences are:

k = 4: 4 2 1

k = 3: 3 10 5 16 8 4 2 1

k = 5: 5 16 8 4 2 1

k = 6: 6 3 10 5 16 8 4 2 1

k = 27: 27 82 41 124 62 31 94 47 142 71 214 107 322 161 484 242 121 364 182 91 274 137 412 206
 103 310 155 466 233 700 350 175 526 263 790 395 1186 593 1780 890 445 1336 668 334 167 502 251

754 377 1132 566 283 850 425 1276 638 319 958 479 1438 719 2158 1079 3238 1619 4858 2429 7288
 3644 1822 911 2734 1367 4102 2051 6154 3077 9232 4616 2308 1154 577 1732 866 433 1300 650
 325 976 488 244 122 61 184 92 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1

The number 27 has the distinction that it is the smallest value of k whose $3k+1$ sequence has at least 100 terms in it. These results were computed by the following C++ program:

```
// Author: Ted Obuchowicz
// Oct. 10, 2002
// file: 3k+1.C

#include <iostream>
using namespace std;

int main()
{
  unsigned long int number = 1;
  unsigned long int term = 1;
  unsigned long int length= 1;
  int done = 0;

  do
  {
    number = number + 1;
    term = number;          // Ahhhh think what this means in terms of
                           // VHDL variables and signals!!!

    length = 1;
    cout << "Testing number = " << number << endl;

    while ( term != 1)  // generate the next term and increment length
    {
      if (term % 2 == 0 )    // term is even
        term = term / 2 ;    // perform in integer division
      else
        term = (3*term + 1);

      cout << " " << term ;
      length = length + 1;
    };
    cout << endl << endl; //
  } while (length < 100);
  done = 1;
  cout << number << endl;
  return 0;
}
```

PART 1: Using a single VHDL clocked process to design the sequential "3k+1" generator in hardware.

Using a single VHDL clocked process, find the smallest positive integer number whose 3k+1 sequence has at least 9 terms in it. It is left as a design consideration as to which of the variables in the provided C++ program are to be mapped to registers in the VHDL code. It is also left as a design consideration as to whether signals and/or variables are to be employed in the VHDL code.

The system is to have a "done" output which indicates to the user that the algorithm has completed and the final answer is available. Once the "done" signal has reached '1', any subsequent clock edges should not affect the contents stored in any of the registers within the hardware system.

The system is to have an asynchronous reset (it is left as a design consideration to determine what the initial reset values of the registers should be). Figure 1 gives typical simulation results. Note that in this figure the values of the various registers are displayed in decimal (as unsigned numbers). The radix used to display an object in the Waveform window can be specified by first selecting the object name with the right mouse button and then choosing one of the possible values (Binary, Octal, Decimal, Unsigned, etc).

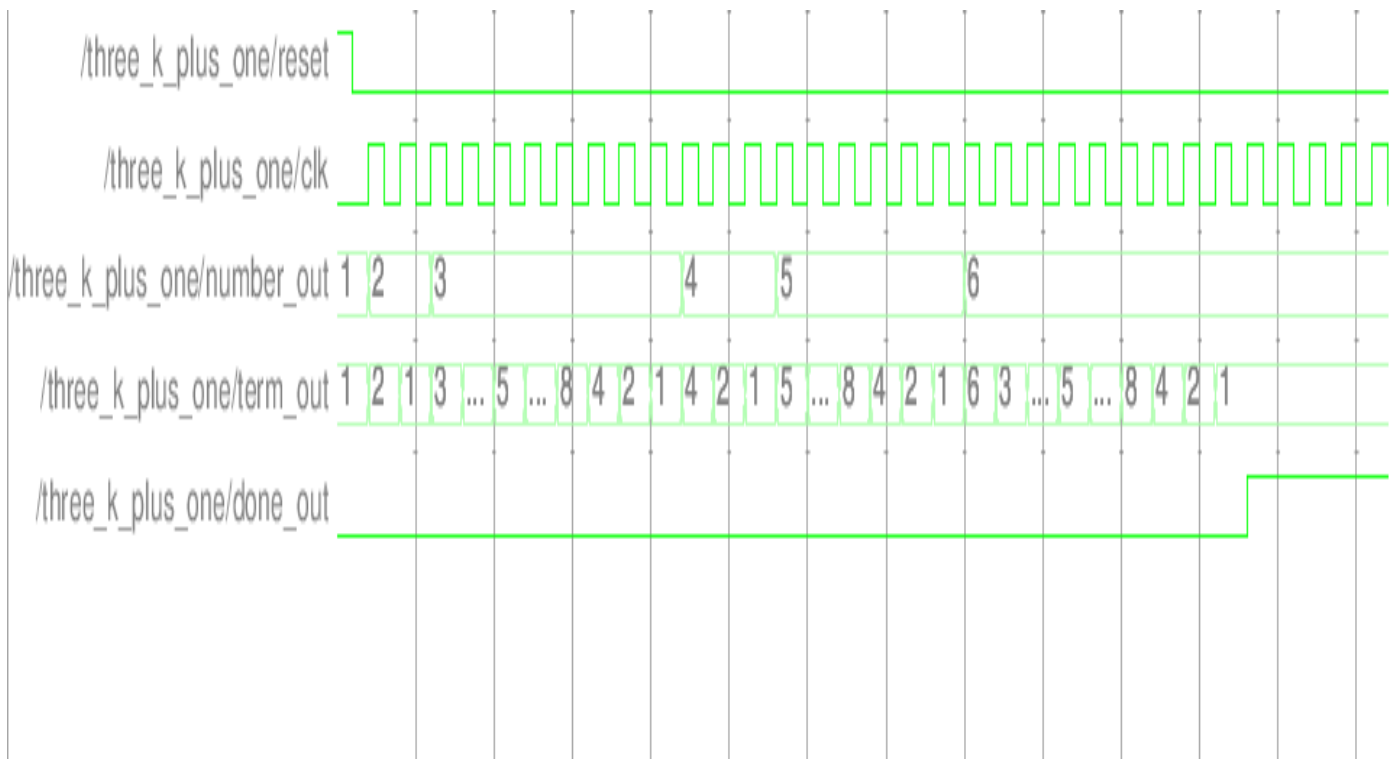


Figure 1: Typical simulation results from the 'one clocked process' 3k+1 sequence generator.

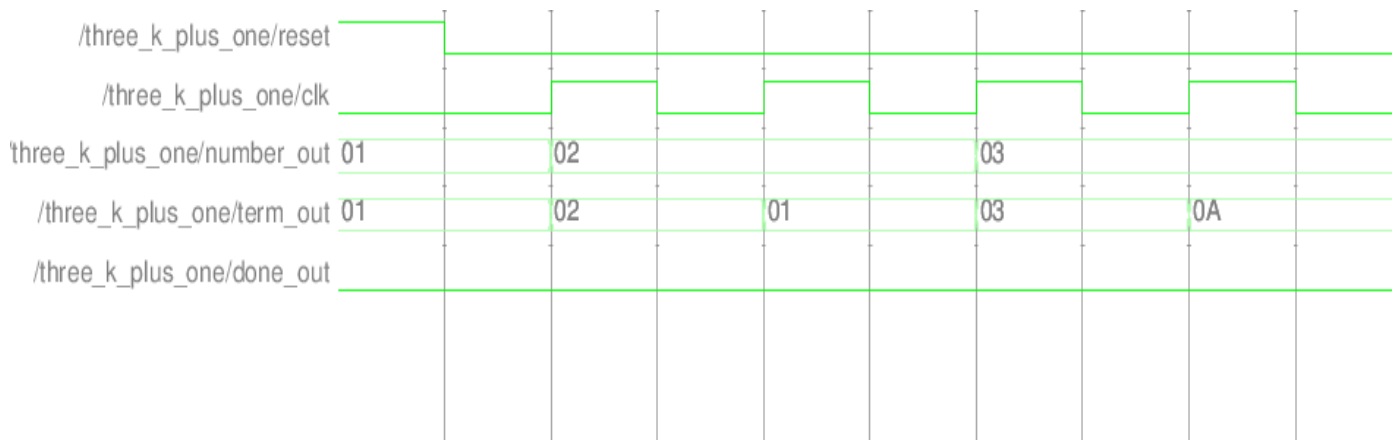


Figure 2: Typical (partial) simulation results from the ‘one clocked process’ with signals displayed with hexadecimal radix.

Design Considerations:

1. The simulation results given previously give the values of internal signals used to hold the values of ‘number_out’ and ‘term_out’. The final project, when implemented on the FPGA board is to use the available 7-segment LED displays on the Nexys A7 FPGA board to display the value of ‘number_out’ on one 7-segment LED display and the value of ‘term_out’ on two 7-segment LED displays. As the value of ‘number_out’ ranges from 0 to 6 (in integer), a single hexadecimal digit is sufficient to display (in hex) the value of number_out on a 7-segment LED display. Similarly, two hexadecimal digits (corresponding to two 7-segment LED displays) is sufficient to display the value of ‘term_out’ as the maximum value of term_out is 16 (10 in hexadecimal). For simplicity, we will simply display the values in hexadecimal and not concern ourselves with the details of converting from binary into binary coded decimal. Recall that any 8 bit binary number representing an unsigned integer can easily be converted into its equivalent two digit hexadecimal format by grouping the bits into groups of 4, and expressing each group of 4 bits as a single hex digit between 0 and A. For example:

$$(41)_{10} = (00101001)_2 = (0010 \ 1001)_2 \\ = (\ 2 \ \ 9 \)_{16}$$

2. Rather than using a pushbutton switch as the clock input, the final project, when implemented on the FPGA board, will make use of the 100 MHz on-board clock oscillator together with the necessary clock-divider hardware to produce the 1 Hz clock output for the datapath and approximately 1000 Hz clock used for the time-multiplexed 7-segment LED driver hardware. It is suggested that the reader first implement working versions of the VHDL code in which the values of the internal signals are displayed in binary (both in simulation) and on the FPGA board (on individual LEDs) using a pushbutton switch for the clock input, and once the design is working, then add the necessary hardware modification for the ‘auto-clocking’ and 7-segment LED display for the values of ‘term_out’ and ‘number_out’.

For the final version, use the following VHDL entity specification:

```
library IEEE;
use ieee.numeric_std.all ; -- needed for "*" and "/" operators
use IEEE.std_logic_1164.all;

entity three_k_plus_one is
port( reset : in std_logic; -- asynchronous
      clk_in  : in std_logic; -- the 100MHz FPGA board clock
      an: out std_logic_vector(7 downto 0 ); -- the 8 anodes of each
                                              -- 7-seg display
      sseg : out std_logic_vector(7 downto 0 );
      done_out      : out std_logic);
end three_k_plus_one;
```

The 'number_out' (an internal signal within the VHDL code) gives the value of the number which is being tested to see whether it's $3k+1$ sequence has at least 9 terms in it. The term_out value (another internal signal in the VHDL code) is used to indicate the various terms of the sequence pertaining to a given number_out as they are being generated. For example:

```
number_out:      3
term_out:        3  10  5   16   8   4   2   1
```

Upon examination of the simulation results in Figure 1 and Figure 2, it is evident that upon every rising clock edge, a new term in the sequence for a given number is generated. Eventually, a number which has at least nine terms is encountered, and the done signal is set to '1'. Subsequent clock pulses have no effect on the system until the user asserts the asynchronous reset.

3. Upon examination of the various values of the numbers contained within the registers, it is evident that registers of size (6 downto 0) will be sufficiently large to hold the largest numbers encountered.

4. The astute reader may have noticed that the provided entity specification makes use of the:

```
library IEEE;
use ieee.numeric_std.all ;
use IEEE.std_logic_1164.all;
```

instead of the usual:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
```

Andrew Rushton, in his text *"VHDL for Logic Synthesis"* (Second Edition, John Wiley and Sons) presents a very concise summary of this package. The following is a condensed version of Rushton's original words:

"The packages are defined by the IEEE standard number 1076.3 (IEEE Design Automation Standards Committee, 1997). The result of the standardization effort is two packages: one called `numeric_bit` and based on arrays with elements of type `bit`; the other called `numeric_std` and based on arrays with elements of type `std_logic`. The `numeric_std` package can be used in tri-state modelling since the data type `std_logic` can be used to model high-impedance values, whereas the `numeric_bit` types cannot. The choice of which package to use depends on whether one writes models using the type `bit`, or whether one uses type `std_logic`. There is no reason to mix the two types and therefore no reason to mix the two packages. Most designers use `std_logic` because it can be used to represent unknown and high impedance values. This means that `numeric_std` is the package most commonly used.

These two packages are found in library `ieee`. Therefore to use the package `numeric_std`, the following declarations will be needed before the entity or architecture that is to use the packages:

```
library IEEE;
use ieee.numeric_std.all ;
use IEEE.std_logic_1164.all;
```

The package `numeric_std` defines two data types, both of which are unconstrained arrays of the element type `std_logic`. The types are called `unsigned` and `signed`. The type `signed` represents 2's complement signed numbers, whilst the type `unsigned` represents unsigned magnitudes.

OPERATORS

The set of operators available for type `signed` is:

```
boolean    not, and, or , nand, nor, xor
comparison = , /=, < , <= , > , >=
arithmetic sign -, abs, + , -, *, / , mod , rem
concatenation &
```

Not all the operators in this list are fully synthesizable. Restrictions apply to the division operators `"/`, `mod`, and `rem`.

(Note added by Ted: The `"/` operator is synthesizable when we are **dividing by a power of two**, since this corresponds to a shift operation)

Type `unsigned` has a slightly more restrictive set of operators, in that the sign operator `"-` and the `abs` operator are not present, since they are meaningless for unsigned arithmetic.

RESIZE FUNCTIONS

The package defines two types of functions, both called `resize` which are used to truncate and extend values of types signed and unsigned". [3]

The use of the `resize` function is most commonly employed when multiplying two values of type unsigned or signed. Consider the following simple example:

```
library IEEE;
use ieee.numeric_std.all ; -- needed for "*" and "/" operators
                           -- and resize()
use IEEE.std_logic_1164.all;

entity mult is
port(  mick : in unsigned(3 downto 0);
      product : out unsigned(3 downto 0));
end mult;

architecture example of mult is

signal term : unsigned(3 downto 0);

begin

    term <= mick;
    product <= term * 3 ; -- this will cause `vcom' errors

end ;

--ted@deadflowers PROJECT_FALL_2025 8:31pm >vcom mult.vhd
-- Compiling entity mult
-- Compiling architecture example of mult
-- ** Error: mult.vhd(19): (vcom-1272) Length of expected is 4;
length
-- of actual is 8.
-- ** Error: mult.vhd(22): VHDL Compiler exiting
-- since term is 4 bits wide, the * operator uses 4 bits for the
constant 3
-- and will generate a product which is now 8 bits wide
-- this causes an error since we cant squeeze 8 bits into 4 bits
-- the solution is to use the `resize' function to truncate
-- the 8 bit product into a signal of 4 bits
-- provided that the product will fit into 4 bits, this will not
```

```
-- affect the result
-- of course, if the product is larger than 4 bits wide, we will
-- lose information
-- SOLUTION IS TO USE:
--resize(3*term, 4))
```

As indicated by the comments in the VHDL code, this will not compile since the resulting product of multiplying a 4 bit number by another 4 bit number is 8 bits, and the code is attempting to squeeze 8 bits of product into a 4 bit signal. The solution is to resize the 8 bit product to only the low order 4 bits by truncating the high order eight bits. This is performed by the `resize()` function:

```
library IEEE;
use ieee.numeric_std.all ; -- needed for "*" and "/" operators
                           -- and for `resize'
use IEEE.std_logic_1164.all;
```

```
entity mult is
port(  mick : in unsigned(3 downto 0);
      product : out unsigned(3 downto 0));
end mult;
```

```
architecture example of mult is
```

```
signal term : unsigned(3 downto 0);
```

```
begin
```

```
    term <= mick;
    product <= resize(term * 3, 4) ;
```

```
--
-- -----
-- | X | X | X |X | K | K | K | K |
-- -----
--
```

```
-- X means the bit is `chopped off'
-- by the resize() function
-- K means the bit is kept as part of the answer
-- high order 4 bits will be `chopped off'
-- and only the low order 4 bits will be kept
-- and assigned to product
```

```
end ;
```


Figure 3 gives some typical simulation results for this code.

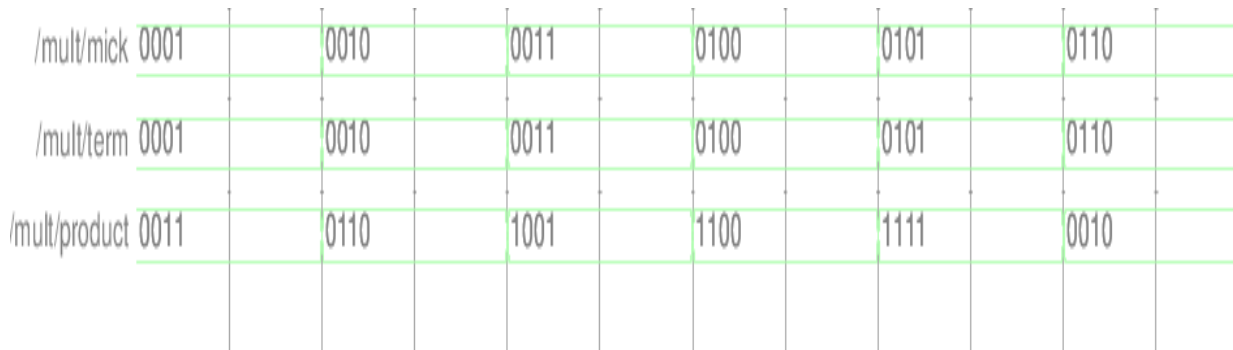


Figure 3: Simulation results with `resize(term*3, 4)`.

Note how in Figure 3, when `mick` has value of 6 (0110), there are not enough bits in the resized product to hold the value of $6 * 3 = 18$.

CONVERTING FROM unsigned (or signed) to `std_logic_vector`

A consequence of the use of the `numeric_std` data types `unsigned` and `signed` is that one cannot directly assign a signal (or variable) of type `unsigned` or `signed` to a signal (or variable) of type `std_logic_vector`. Attempting to do so would result in a compile time error as shown by the following example VHDL code:

```
library IEEE;
use ieee.numeric_std.all ; -- needed for "*" and "/" operators
use IEEE.std_logic_1164.all;
```

```
entity example is
port(  mick : in unsigned(3 downto 0);
      keith : out std_logic_vector(3 downto 0));
end example;
```

```
architecture bad of example is
```

```
begin
```

```
keith <= mick ;
```

```
end bad ;
```

```
ted@deadflowers PROJECT_SUMMER_2026 7:00pm >vcom
unsigned_to_stdlogicvector.vhd
```

```

Model Technology ModelSim SE-64 vcom 6.6g Compiler 2012.05 May 23
2012
-- Loading package standard
-- Loading package std_logic_1164

-- Loading package numeric_std
-- Compiling entity example
-- Compiling architecture bad of example
** Error: unsigned_to_stdlogicvector.vhd(15): Signal "mick" is type
ieee.numeric_std.unsigned; expecting type
ieee.std_logic_1164.std_logic_vector.
** Error: unsigned_to_stdlogicvector.vhd(17): VHDL Compiler exiting

```

Fortunately, the `numeric_std` package contains conversion (actually casting) functions which can be used as in:

```

architecture good of example is

begin

keith <= std_logic_vector(mick) ;

end good ;

```

The Nexys A7 FPGA board and it's 7-segment LED displays:

Refer to the `Nexys_A7_Reference_Manual_from_digilent_website_as_html.pdf` document available in the `/home/t/ted/PUBLIC/COEN313/TUTORIAL` for an explanation of how the eight 7-segment LED displays need to be multiplexed over time in order to display different numbers on different 7-segment displays. Pong P. Chu in his excellent text *"FPGA Prototyping by VHDL Examples Xilinx Spartan-3 Version"*, Wiley Interscience, ISBN 978-0-470-18531-5, pp. 88-95 provides a nice overview of time-multiplexing four 7-segment LED displays. The method shown can easily be extended to the 7-segment LED displays used in the Nexys A7 FPGA board. Alternatively, AI can be used to **understand** the technique, and then to write one's own VHDL code based upon this understanding. **Do not use any form of any AI to simply make use of any AI generated code for this project.**

Hints and Suggestions:

Do not use the while loop statement within the clocked process. A while loop may be simulated, but, in general, is not synthesizable. For example, consider the following VHDL code:

```

signal num : std_logic_vector(7 downto 0);

begin

process(clk, reset)
begin
    if reset = '1' then
        num <= (others => '0');
    elsif clk'event and clk = '1' then
        while ( num < 256 ) loop
            num <= num + 1;
        end loop;
    end if;
end process;

```

Attempting to synthesize this code with the Precision RTL synthesis tool results in the following error:

Error: [46292]: Module work.while_loop(rtl) cannot be compiled because it contains non-rtl constructs
. Please check the log for warnings or errors about non-synthesizable constructs in this module.

Oddly enough, the `-check_synthesis` option of `vcom` reports that nothing is amiss with this attempted use of a while loop for synthesis:

```

ted@deadflowers Code 7:26pm >vcom -check_synthesis while.vhd
Model Technology ModelSim SE vcom 6.6d Compiler 2010.11 Nov  1 2010
-- Loading package standard
-- Loading package std_logic_1164
-- Loading package std_logic_arith
-- Loading package std_logic_unsigned
-- Compiling entity while_loop
-- Compiling architecture rtl of while_loop

```

Similarly, Xilinx Vivado reports a synthesis error:

INFO: [Synth 8-638] synthesizing module 'bad'
[/nfs/home/t/ted/Coen414/PROJECT_FALL_2025/while.vhd:11]
ERROR: [Synth 8-3380] loop condition does not converge after 2000 iterations
[/nfs/home/t/ted/Coen414/PROJECT_FALL_2025/while.vhd:22]
ERROR: [Synth 8-285] failed synthesizing module 'bad'
[/nfs/home/t/ted/Coen414/PROJECT_FALL_2025/while.vhd:11]

Instead of attempting to directly convert the C++ code into its equivalent VHDL, one may use a clocked process which can be considered to be the "while loop". A clocked process may be considered

an infinite process which is activated with every clock edge - there is no need to directly code the while loops contained in the C++ code into VHDL while loops. Use `if` statements to express the control of the algorithm. Under which condition is the done flip-flop to be loaded with '1'? Under which condition is the value stored in the number register to be incremented? Under which condition are we to generate the next term by dividing the current term by 2? When is the sequence terminated? The proper answers to these questions will give rise to the necessary VHDL code. Finally, in binary, it is very simple to determine whether a given unsigned integer is even or odd. It is so simple that one does not need to use the rem operator...

Requirements:

Simulate and synthesize your design to the Nexys A7 FPGA board. Use the following XDC file:

```
## Clock signal
set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { clk_in }];

##Switches  pushbutton centre
set_property -dict { PACKAGE_PIN N17     IOSTANDARD LVCMOS33 } [get_ports { reset }];

set_property -dict { PACKAGE_PIN V11 IOSTANDARD LVCMOS33 } [ get_ports { done_out } ] ;

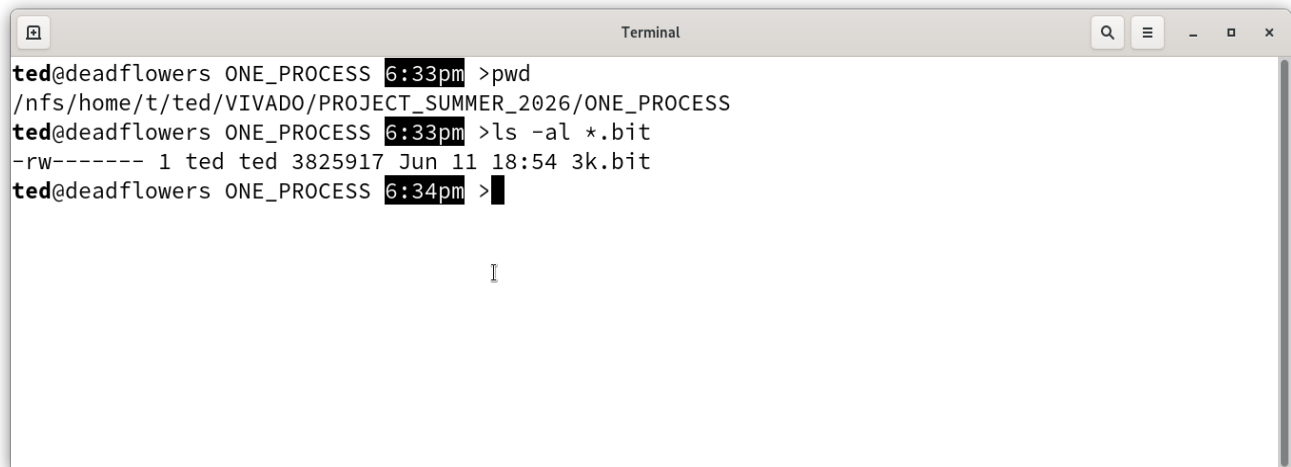
##7 segment display
set_property -dict { PACKAGE_PIN T10     IOSTANDARD LVCMOS33 } [get_ports { sseg[7] }];
set_property -dict { PACKAGE_PIN R10     IOSTANDARD LVCMOS33 } [get_ports { sseg[6] }];
set_property -dict { PACKAGE_PIN K16     IOSTANDARD LVCMOS33 } [get_ports { sseg[5] }];
set_property -dict { PACKAGE_PIN K13     IOSTANDARD LVCMOS33 } [get_ports { sseg[4] }];
set_property -dict { PACKAGE_PIN P15     IOSTANDARD LVCMOS33 } [get_ports { sseg[3] }];
set_property -dict { PACKAGE_PIN T11     IOSTANDARD LVCMOS33 } [get_ports { sseg[2] }];
set_property -dict { PACKAGE_PIN L18     IOSTANDARD LVCMOS33 } [get_ports { sseg[1] }];
set_property -dict { PACKAGE_PIN H15     IOSTANDARD LVCMOS33 } [get_ports { sseg[0] }];
set_property -dict { PACKAGE_PIN J17     IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
set_property -dict { PACKAGE_PIN J18     IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
set_property -dict { PACKAGE_PIN T9      IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
set_property -dict { PACKAGE_PIN J14     IOSTANDARD LVCMOS33 } [get_ports { an[3] }];
set_property -dict { PACKAGE_PIN P14     IOSTANDARD LVCMOS33 } [get_ports { an[4] }];
set_property -dict { PACKAGE_PIN T14     IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
set_property -dict { PACKAGE_PIN K2      IOSTANDARD LVCMOS33 } [get_ports { an[6] }];
set_property -dict { PACKAGE_PIN U13     IOSTANDARD LVCMOS33 } [get_ports { an[7] }];
```

NOTE: For convenience, this XDC file is available on the course Moodle page and in the PUBLIC directory.

Include in your written report:

- the VHDL code
- complete and legible simulation results (no screen captures). In your simulation results, show the values of your signals used for 'term' and 'number' as this conveys more information than the an and sseg ports used to drive the seven segment LEDs.

- a brief, word summary of the VHDL code.
- a legible screen capture of your Linux terminal listing (`ls -al`) indicating the timestamp information of your .bit file and your login name (use the `whoami` command). Refer to Figure 4.



```
ted@deadflowers ONE_PROCESS 6:33pm >pwd
/nfs/home/t/ted/VIVADO/PROJECT_SUMMER_2026/ONE_PROCESS
ted@deadflowers ONE_PROCESS 6:33pm >ls -al *.bit
-rw----- 1 ted ted 3825917 Jun 11 18:54 3k.bit
ted@deadflowers ONE_PROCESS 6:34pm >
```

Figure 4: Screen capture showing directory contents.

Submission Procedure:

The written report is to be submitted by the due date by placing it in the appropriate box which will be located in EV5.139.

The .bit file and the .vdi log file is to be submitted using the Moodle Project submission link. Use the Linux `cp` (copy) command to rename your .bit file as `process.bit` and rename your .vdi file as `process.vdi`.

Part 2: Design of the '3k+1' sequence generator using the ASM chart control unit/datapath methodology.

A datapath/control unit will be designed to perform the same task as done in Part 1. The algorithmic state machine (ASM) chart method will be used to design the system. The control unit will be implemented as a finite-state machine. Figure 5 gives a suggested partial datapath; you are to complete the missing details.

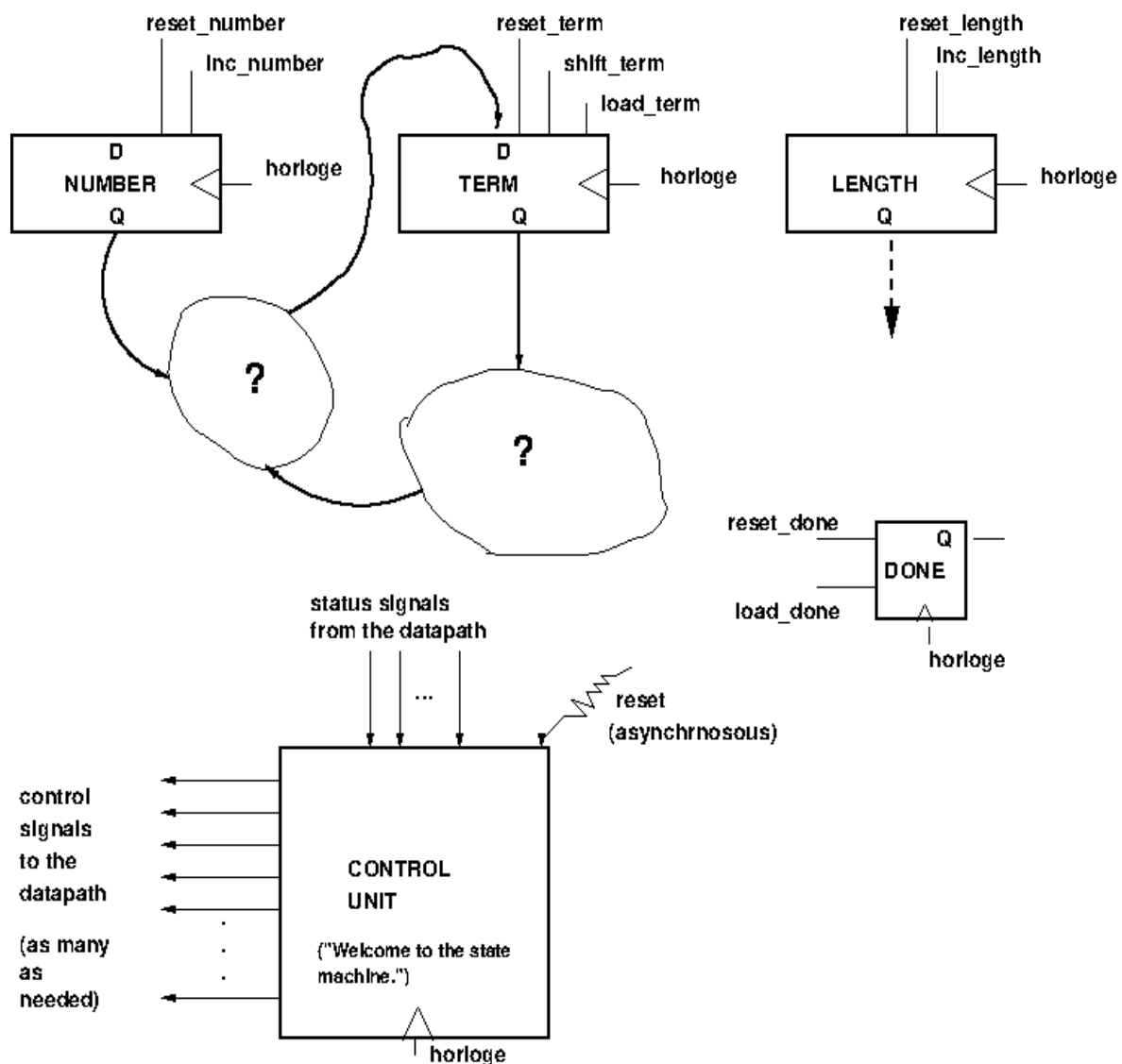


Figure 5: Partial datapath for the 3k+1 sequence generator.

Note: The missing details in the datapath may be both in the form of extra control signals and additional hardware in the datapath.

Design an ASM chart to implement the algorithm expressed by the C++ program. The design choice of whether it is to be a Moore style ASM chart or a Mealy style ASM chart is up to you.

Use the **same** entity specification as was given in Part 1 together with the same XDC file. As with Part 1, for the FPGA board implementation, you are to use the 100 MHz on-board clock together with the 7-segment LED displays to display the values (in hex) of the internal signals 'number' and 'term'. It is suggested that one first works on the basic design without the on board clocking and the 7-segment LED displays to first obtain a working minimal system. This minimal working system can then be modified to include the clocking circuitry and LED displays.

Implement the datapath and control unit using VHDL processes. The processes and internal signals are to implement the datapath and the interconnections between the various hardware elements contained in the datapath. For example, the counter called length in the datapath given in Figure 5 can be coded with the following process:

```
architecture keith of three_k_plus_one is

signal length      : unsigned(3 downto 0);

begin

-- the register holding the length

process(clk)
begin
    if clk'event and clk = '1' then
        if reset_length = '1' then
            length <= "0001";
        elsif inc_length = '1' then
            length <= length + "0001";
        end if;
    end if;
end process;
```

Your VHDL code should consist of only processes (with the possible exception of using concurrent signal assignment statements to drive the output ports, or some concurrent statements for the 7-segment control circuitry). If desired, you may use port map statements to create a top-level entity consisting of an instance of a 'datapath' component and an instance of a 'control unit' component.

Hint: Since we are only dividing even numbers by 2, this division operation can easily be performed in hardware by simply shifting the number to be divided by one bit to the right.

Requirements:

Simulate and synthesize your design to the Nexys A7 FPGA board.

Submit the following together with your written lab report:

- The completed ASM chart and the completed block diagram of your datapath/control unit. Indicate clearly in your block diagram all the control signals and status signals used in the system. Recall that control signals are outputs of the control unit which are used to control the datapath (i.e. load a register at the specific point in time with some specific data) and the status signals are outputs of the datapath used by the control unit to determine the next state it should go to at the next clock edge.
- The VHDL code for the complete system.
- Simulation results as produced by Modelsim. Trace through the sequence of states your control unit progresses through and show the control signals produced by the control unit during each clock cycle as well as the values of the data contained in the datapath. It is suggested you make use to the "Custom Range" to ensure legibility in your simulation results. Refer to Figures 6 and 7.

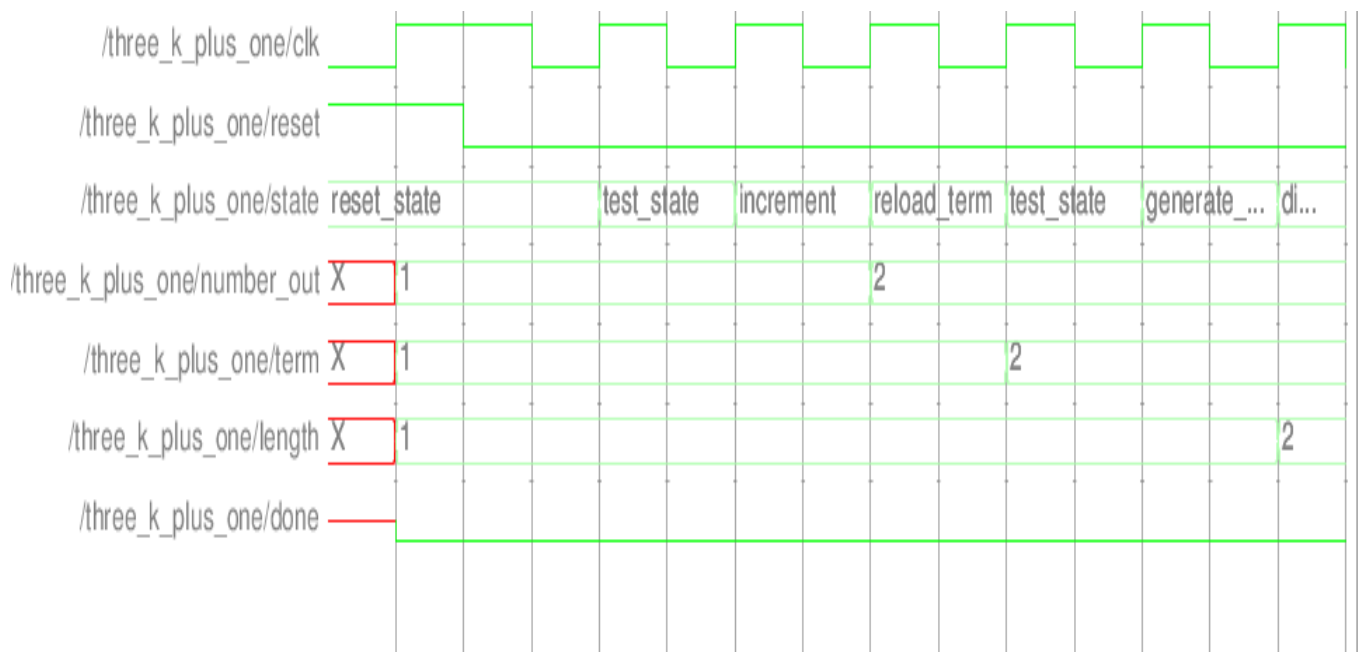


Figure 6: 0 ns - 30 ns range simulation results of ASM chart datapath/control unit design.

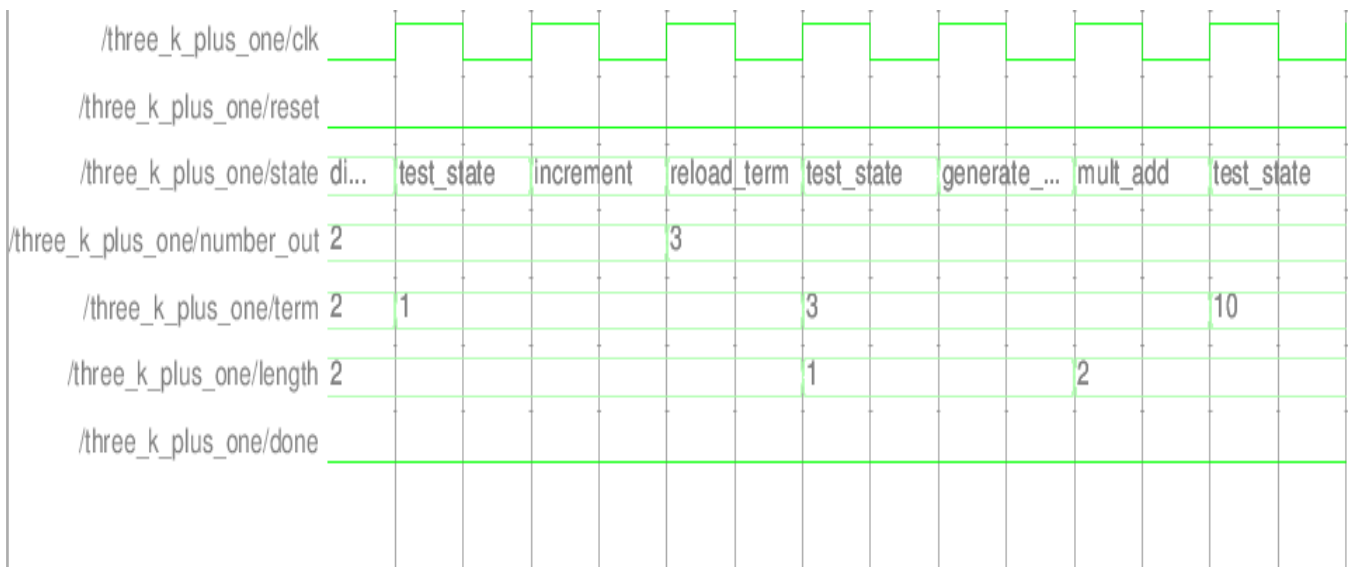


Figure 7 : 30 ns - 60 ns range simulation results of ASM chart datapath/control unit design.

The simulation results for the ASM method may be included within an Appendix section of your report consisting of full-page print outs of the waveforms.

- a screen capture of your Linux terminal listing (ls -al) indicating the timestamp information of your .bit file and your login name (use the whoami command).

Submission Procedure:

The written report is to be submitted by the due date by placing it in the appropriate box which will be located in EV5.139.

The .bit file and the .vdi log file is to be submitted using the Moodle Project submission link. Use the Linux cp (copy) command to rename your .bit file as asm.bit and rename your .vdi file as asm.vdi.

A note on the usefulness of log files:

The markers will be consulting the submitted log files to search for any errors/warnings. It is your responsibility as VHDL hardware designers to ensure your design is correct by reviewing the log files post synthesis/implementation. In addition to listing any warnings and errors, the log files contain useful information about the design. For example, the number of inferred registers and their size (in terms of number of bits) for my implementation of the 'one clocked process' version is listed in a log file as:

```
-----
Start RTL Component Statistics
-----
```

```
Detailed RTL Component Info :
```

```

+---Adders :
      2 Input      7 Bit      Adders := 2
      2 Input      4 Bit      Adders := 1
+---Registers :
              7 Bit      Registers := 2
              4 Bit      Registers := 1
              1 Bit      Registers := 1
+---Muxes :
      2 Input      7 Bit      Muxes := 2
      2 Input      4 Bit      Muxes := 1
      2 Input      1 Bit      Muxes := 3

```

Finished RTL Component Statistics

Whereas, for my `ASM datapath/control unit design', the number of RTL components is listed the corresponding log file as:

Start RTL Component Statistics

Detailed RTL Component Info :

```

+---Adders :
      2 Input      7 Bit      Adders := 1
      2 Input      4 Bit      Adders := 1
+---Registers :
              7 Bit      Registers := 2
              4 Bit      Registers := 1
              3 Bit      Registers := 1
              1 Bit      Registers := 1
+---Muxes :
      4 Input      7 Bit      Muxes := 1
      3 Input      7 Bit      Muxes := 1
      2 Input      7 Bit      Muxes := 2
      2 Input      3 Bit      Muxes := 1
      9 Input      3 Bit      Muxes := 1
      8 Input      1 Bit      Muxes := 12

```

The designer should verify that the synthesized number of registers is the same as the number of registers the designer had wanted when the RTL VHDL code was first written.

References:

1. http://en.wikipedia.org/wiki/Collatz_conjecture

2. C++ code written by T. Obuchowicz based upon the pseudocode algorithm presented by Dr. Peter Grogono in the course COMP 244 Programming Methodology, Winter 1987, Concordia University.
3. *VHDL for Logic Synthesis*, Second Edition, Andrew Rushton, ISBN 0-47198325-X, 1998, pp.123-129.

T. Obuchowicz

Fri 12 Jun 2026 07:00:14 PM EDT